Lista de Exercícios: Jogo de Monstros Herança, Polimorfismo, Classes Abstratas e Encapsulamento

Prof. Gabriel Rodrigues Caldas de Aquino

Compilado em:
September 14, 2026

1 Introdução

Nesta lista vocês vão construir um pequeno jogo de batalha entre monstros inspirado em Pokémon, mas com uma regra simples: cada monstro é de um tipo entre **Pedra**, **Papel** ou **Tesoura**. Assim como no jogo clássico, um tipo tem vantagem sobre o outro:

- **Pedra** tem vantagem sobre **Tesoura**
- **Tesoura** tem vantagem sobre **Papel**
- **Papel** tem vantagem sobre **Pedra**

Quando um monstro ataca outro que ele tem vantagem, o dano é **dobrado**. Quando ataca outro que o tem vantagem, o dano é **reduzido pela metade**. Quando ataca um do mesmo tipo, o dano é normal (sem nenhum multiplicador).

Além disso, os monstros podem carregar **itens** que os ajudam na batalha: um item de cura e um item de dano. Cada monstro só pode carregar **no máximo 2 itens**.

O objetivo é praticar os conceitos de **herança**, **polimorfismo**, **classes abstratas** e **encapsulamento**.

2 Requisitos Gerais

1. Todos os atributos devem ter **getters** e **setters** públicos.
2. Os setters devem conter validações sempre que fizer sentido (por exemplo, vida entre 0 e 100).
3. A classe `Monstro` deve ser **abstrata** e definir um método abstrato que será implementado pelas classes filhas.

4. A classe `Item` também deve ser **abstrata**.
5. Utilize **polimorfismo**.

3 Monstros

Crie a classe abstrata `Monstro` com os seguintes atributos:

- `nome`
- `vida` (deve sempre estar entre 0 e 100)
- `danoBase`
- `itens` (lista com no máximo 2 itens)

A classe `Monstro` deve ter:

1. Getters e setters públicos para todos os atributos.
2. O setter de `vida` deve garantir que o valor fique sempre entre 0 e 100.
3. Um método `esta_vivo()` que retorna `True` se a vida for maior que 0.
4. Um método `adicionar_item(item)` que só adiciona se ainda houver espaço (no máximo 2 itens) e se o objeto for realmente um `Item`.
5. Um método `usar_item(indice, alvo)` que usa o item da posição indicada, remove-o da lista após o uso e valida se o índice é válido.
6. Um método `atacar(alvo)` que calcula o dano e reduz a vida do alvo. Esse método deve impedir que monstros mortos ataquem.
7. Um método **abstrato** chamado `calcular_dano(alvo)` que cada subclasse vai implementar com a sua própria regra de vantagem.

3.1 Subclasses de Monstro

Crie as classes `MonstroPedra`, `MonstroPapel` e `MonstroTesoura`, todas herdando de `Monstro`. Cada uma deve:

1. Implementar o método `calcular_dano(alvo)` de acordo com a regra de vantagem:
   - Se o monstro vence o alvo, o dano é `danoBase` $\times$ 2.
   - Se o monstro perde para o alvo, o dano é `danoBase` $\times$ 0.5.
   - Se empatam, o dano é apenas `danoBase`.
2. Ter um método `exibir_detalhes()` que mostre o nome, o tipo, a vida atual e quantos itens o monstro está carregando.

4 Itens

Crie a classe abstrata `Item` com os atributos privados:

- `nome`
- `descricao`

A classe `Item` deve ter:

1. Getters e setters públicos.
2. Um método **abstrato** chamado `usar(alvo)` que será implementado pelas subclasses.

4.1 Subclasses de Item

Crie as classes `ItemCura` e `ItemDano`, ambas herdando de `Item`.

1. `ItemCura` deve ter um atributo privado `valorCura` e, ao ser usado, aumentar a vida do `usuario` respeitando o limite máximo de 100.
2. `ItemDano` deve ter um atributo privado `valorDano` e, ao ser usado, reduzir a vida do `alvo`. Se nenhum alvo for passado, o item não deve ser usado.
3. Ambas devem ter getters e setters públicos para seus atributos específicos.

5 Testando o Jogo

Depois de implementar tudo, faça o seguinte no seu notebook:

1. Crie pelo menos um monstro de cada tipo.
2. Crie alguns itens de cura e de dano.
3. Adicione itens aos monstros, testando o limite de 2 itens.
4. Faça um monstro atacar o outro e mostre o dano causado.
5. Faça um monstro usar um item e mostre o efeito.
6. Coloque todos os monstros em uma lista e percorra essa lista chamando `exibir_detalhes()` para demonstrar polimorfismo.

6 Perguntas para Reflexão

1. Por que a classe `Monstro` precisa ser abstrata?
2. O que aconteceria se o método `calcular_dano` não fosse abstrato?
3. Como o polimorfismo aparece quando chamamos `atacar` em monstros diferentes?
4. Por que é importante que o setter de vida limite o valor entre 0 e 100?